

Hands-On MIS Projects

Chapter 1 - Information Systems in Global Business Today

Parsa Mohebian

July 8, 2026

Contents

1	Introduction	3
2	Management Decision Problem 1-7: Snyder's of Hanover	3
2.1	Problem Summary	3
2.2	Impact on Business Performance	3
2.3	Impact on Management Decision Making	3
2.4	Recommended Information System Solution	3
3	Management Decision Problem 1-8: Dollar General	4
3.1	Problem Summary	4
3.2	Decisions Before Investing in an Information System	4
4	Database Analysis Project 1-9	5
4.1	Dataset Description	5
4.2	Data Preparation	5
4.3	Sales Analysis Queries	6
4.4	Restocking Analysis	7
4.5	Promotional Campaign Analysis	7
4.6	Pricing and Discount Timing Analysis	8
5	Job Search Project 1-10	9
5.1	Research Method	9
5.2	Job 1: Accounting or Finance Analyst	9
5.3	Job 2: Sales Operations Analyst	10
5.4	Job 3: HRIS or Workday Analyst	10
5.5	Preparation for These Jobs	11
5.6	Summary of Job Search Findings	11
5.7	Sources Consulted	11

6 Conclusion	12
A Appendix A: SQLite to CSV Export Script	13
B Appendix B: Database Profile and Base View SQL	15
C Appendix C: Restocking Analysis SQL	18
D Appendix D: Promotional Campaign Analysis SQL	20
E Appendix E: Pricing and Discount Timing SQL	24

1 Introduction

This report analyzes management information system problems and demonstrates how information systems improve decision making using transactional sales data.

2 Management Decision Problem 1-7: Snyder's of Hanover

2.1 Problem Summary

Snyder's financial department depended heavily on spreadsheets and manual processes for collecting and reporting financial information. Every month, the financial analyst collected spreadsheets from more than 50 departments worldwide, consolidated the data, and created the company's profit-and-loss statement manually. This process was slow, difficult to maintain, and vulnerable to errors.

2.2 Impact on Business Performance

The spreadsheet-based system negatively affected business performance.

- The financial analyst spent significant time collecting and consolidating spreadsheets instead of performing financial analysis.
- Manual data entry increased the possibility of errors and inconsistent reports.
- Financial information was delayed because reports required manual preparation.
- The company faced higher operational costs because employees performed repetitive administrative tasks.
- The system was difficult to scale as the company expanded globally.

2.3 Impact on Management Decision Making

The existing system limited management decision making because managers did not have fast access to accurate and updated financial information.

Delayed reporting made it harder to identify financial problems, compare department performance, and respond quickly to changes in business conditions. Incorrect or outdated information could result in poor decisions regarding budgeting, production, and resource allocation.

2.4 Recommended Information System Solution

Snyder's should implement an integrated financial information system based on a centralized database. The system should allow departments to enter financial data directly into a shared platform and automatically generate standardized reports.

The system should include automated reporting, security controls, and management dashboards. An Enterprise Resource Planning (ERP) system could integrate financial data with other business functions such as sales, inventory, and production.

This solution would reduce manual work, improve reporting accuracy, provide faster access to information, and support better management decisions.

3 Management Decision Problem 1-8: Dollar General

3.1 Problem Summary

Dollar General operates discount stores where minimizing costs is essential. However, the company does not have an automated inventory tracking system at each store. Managers cannot accurately verify incoming deliveries, which has resulted in inventory errors and increasing merchandise losses.

The company needs to determine whether an information system investment can improve inventory accuracy, reduce losses, and support better operational decisions.

3.2 Decisions Before Investing in an Information System

Before investing in an inventory management system, Dollar General should evaluate several important factors.

- **Identify business goals:** The company should define the problems it wants to solve, such as reducing inventory losses, improving inventory accuracy, and ensuring product availability.
- **Determine required system features:** The company should decide whether it needs barcode scanning, real-time inventory tracking, automatic stock updates, inventory alerts, and reporting tools.
- **Evaluate costs and benefits:** Management should compare implementation costs, including hardware, software, and training, against expected benefits such as lower losses and improved operational efficiency.
- **Plan implementation:** The company must decide how the system will be introduced, how employees will be trained, and how existing inventory data will be transferred.
- **Consider security requirements:** The company should establish rules for protecting inventory data and controlling access to the system.

Dollar General should implement an automated inventory management system using barcode scanning and a centralized database. This system would provide accurate inventory information, reduce merchandise losses, improve replenishment decisions, and increase overall operational efficiency.

4 Database Analysis Project 1-9

4.1 Dataset Description

The project uses a transactional marketplace database stored in SQLite format. The database contains raw sales transaction data that can be used to analyze product demand, regional sales performance, seller performance, and monthly sales trends.

The database contains five main tables:

- **orders**: contains order identification numbers, order dates, and customer contact information.
- **order_items**: contains the products sold in each order, seller identifiers, product identifiers, and item prices.
- **products**: contains product identifiers, product categories, and product weight information.
- **sellers**: contains seller identifiers and seller state information.
- **order_reviews**: contains customer review scores and review comments.

The main analysis used the **orders**, **order_items**, **products**, and **sellers** tables. The **order_reviews** table was not required for the three main sales trend questions, but it could be used in future analysis to study customer satisfaction.

The dataset included 96,478 orders, 110,197 order item records, 32,216 products, 2,970 sellers, and 39,462 reviews. The order dates ranged from September 15, 2016 to August 29, 2018.

Table	Number of Records
orders	96,478
order_items	110,197
products	32,216
sellers	2,970
order_reviews	39,462

Table 1: Database Tables and Record Counts

4.2 Data Preparation

The original dataset was provided as a SQLite database file. Because the project requires database querying and reporting, SQLite was used to inspect the database structure and run SQL queries. The database was also exported to CSV files so the tables could be opened in spreadsheet or database software if needed.

A Python script was created to export each SQLite table into a separate CSV file. The exported files were:

- `orders.csv`
- `order_items.csv`
- `products.csv`
- `sellers.csv`
- `order_reviews.csv`

The main sales analysis was performed by joining the order, product, and seller tables into a single SQL view named `sales_analysis_base`. This view created a structure similar to the textbook sales database.

Textbook Field	Dataset Field Used
Store identification number	<code>seller_id</code>
Sales region	<code>seller_state</code>
Item number	<code>product_id</code>
Item description	<code>product_category_name</code>
Unit price	<code>price</code>
Units sold	Count of records in <code>order_items</code>
Sales period	Month extracted from <code>timestamp</code>

Table 2: Mapping Textbook Sales Fields to Dataset Fields

A missing data check was also performed on the joined sales view. The analysis found no missing product categories, seller states, or prices in the final analysis table.

4.3 Sales Analysis Queries

SQL queries were used to answer the three management questions in the assignment. The queries joined the transactional tables and summarized the data by product category, seller region, seller, and month.

The first query group identified which products should be restocked by ranking product categories and individual products by units sold and total revenue. Since the dataset did not include inventory quantity, high sales volume was used as a proxy for high customer demand.

The second query group identified stores and regions that may benefit from promotional campaigns. In this dataset, `seller_id` represented the store or seller, and `seller_state` represented the sales region. Regions with low revenue and low sales volume were treated as candidates for additional marketing support.

The third query group analyzed monthly sales trends. Since the dataset did not include a direct discount field, monthly revenue and units sold were used to identify stronger and weaker sales periods. Stronger months were considered better candidates for full-price selling, while weaker months were considered better candidates for discounts or promotional campaigns.

4.4 Restocking Analysis

The first management question asks which products should be restocked. The dataset does not include current inventory levels or stock quantities. Therefore, units sold were used as a proxy for customer demand. Product categories with the highest sales volume were treated as the strongest candidates for restocking.

The analysis showed that the highest-demand product categories were `cama_mesa_banho`, `beleza_saude`, `esporte_lazer`, `moveis_decoracao`, and `informatica_acessorios`.

Product Category	Units Sold	Total Revenue
<code>cama_mesa_banho</code>	10,953	1,023,434.76
<code>beleza_saude</code>	9,465	1,233,131.72
<code>esporte_lazer</code>	8,431	954,852.55
<code>moveis_decoracao</code>	8,160	711,927.69
<code>informatica_acessorios</code>	7,644	888,724.61

Table 3: Top Product Categories by Units Sold

Based on this analysis, managers should prioritize restocking these high-demand categories. The category `beleza_saude` is especially important because it has both high sales volume and the highest total revenue among the top categories. This suggests that keeping this category available could have a significant impact on sales performance.

4.5 Promotional Campaign Analysis

The second management question asks which stores and sales regions would benefit from a promotional campaign and additional marketing. In this dataset, `seller_id` was used as the store identifier, and `seller_state` was used as the sales region.

The analysis compared sales performance by region using units sold, number of sellers, total revenue, and average price. Regions with low total revenue were identified as possible candidates for promotional campaigns or additional marketing support.

Region	Units Sold	Number of Sellers	Total Revenue
AM	3	1	1,177.00
PA	8	1	1,238.00
SE	10	2	1,606.20
PI	11	1	2,383.00
RO	14	2	4,762.20
MS	50	5	8,551.69
RN	56	5	9,992.60
PB	37	6	16,395.50
MT	144	4	17,007.82
CE	90	12	19,517.84

Table 4: Lowest-Performing Sales Regions by Revenue

The regions AM, PA, SE, PI, and RO had the lowest total revenue. These regions may benefit from additional marketing campaigns because their sales activity is much lower than the strongest regions.

However, the results should be interpreted carefully. Some of these regions have only one or two sellers, which suggests that low revenue may be caused by weak market presence rather than only weak customer demand. Therefore, management should consider both promotional campaigns and seller expansion strategies in these regions.

For comparison, the strongest region was SP, with 78,604 units sold and 8,509,511.46 in total revenue. This shows a large performance gap between SP and the lower-performing regions.

4.6 Pricing and Discount Timing Analysis

The third management question asks when products should be offered at full price and when discounts should be used. The dataset does not contain a direct discount field, so monthly sales volume and total revenue were used to identify stronger and weaker sales periods.

High-demand months are better candidates for full-price selling because customers are already purchasing more products. Low-demand months are better candidates for discounts, promotions, or additional marketing because sales activity is weaker.

Month	Units Sold	Total Revenue
May	11,814	1,466,882.94
August	11,939	1,393,276.34
July	11,379	1,349,557.98
April	10,396	1,314,203.77
March	10,914	1,312,555.10

Table 5: Strongest Months by Total Revenue

Based on this result, managers should consider selling products closer to full price during March, April, May, July, and August. These months had the strongest sales performance and may not require aggressive discounting.

Month	Units Sold	Total Revenue
September	4,740	607,534.64
October	5,527	688,572.76
December	6,188	726,044.09
November	8,475	987,765.37
January	8,950	1,036,443.36

Table 6: Weakest Months by Total Revenue

Discounts and promotional campaigns should be considered during September, October, December, November, and January because these months had weaker sales performance. Promotions during these periods may help increase demand and improve revenue.

The results should be interpreted carefully because the dataset does not cover all calendar months equally. The data begins in September 2016 and ends in August 2018, so some periods may be incomplete. Even with this limitation, the monthly trend provides useful guidance for pricing and promotional planning.

5 Job Search Project 1-10

5.1 Research Method

For this part of the project, I reviewed current job-posting results for business positions in accounting, finance, sales, marketing, and human resources. The goal was to find jobs that are not purely technical IT jobs, but still require information systems knowledge. The selected jobs show that information systems skills are now important across many business functions.

The research focused on three representative roles:

- Accounting or finance analyst roles
- Sales operations analyst roles
- Human resources information systems analyst roles

5.2 Job 1: Accounting or Finance Analyst

One job category that requires information systems knowledge is accounting and financial analysis. Current job postings for accounting and finance analyst roles commonly require experience with ERP systems, accounting systems, advanced Excel, large datasets, and business reporting tools.

For example, an Accounting Transaction Analyst posting listed experience with ERP or accounting systems, comfort working with large datasets, and advanced Excel skills as important qualifications. Other financial analyst postings also mention SQL, Tableau, budgeting systems, and financial automation.

The information systems knowledge required for this type of job includes:

- Understanding ERP and accounting information systems
- Using spreadsheet tools such as Excel for financial analysis
- Working with large datasets
- Creating reports for budgeting, forecasting, and performance analysis
- Understanding data accuracy and internal controls

This role connects directly to management information systems because financial decisions depend on accurate, timely, and well-organized data. Instead of only recording financial transactions, accounting and finance employees are expected to use systems to analyze data and support decision making.

5.3 Job 2: Sales Operations Analyst

A second job category requiring information systems knowledge is sales operations. A Sales Operations Analyst posting required familiarity with SAP, CRM systems, SQL, Salesforce, Power BI, spreadsheet analysis, and sales reporting.

The information systems knowledge required for this type of job includes:

- Understanding customer relationship management systems such as Salesforce
- Using ERP or SAP data for sales and operations reporting
- Writing or understanding SQL queries
- Creating dashboards in Power BI or similar business intelligence tools
- Analyzing sales performance, territory performance, and customer data

This role shows how information systems support sales strategy. Sales managers need accurate data about customers, products, territories, and revenue. A sales operations analyst uses information systems to turn raw sales data into reports that help managers improve productivity, identify weak regions, and make better marketing or sales decisions.

5.4 Job 3: HRIS or Workday Analyst

A third job category requiring information systems knowledge is human resources information systems analysis. HRIS Analyst and Workday Analyst postings require knowledge of Workday HCM, HRIS configuration, system administration, reporting, dashboards, data accuracy, security, and process improvement.

The information systems knowledge required for this type of job includes:

- Understanding HRIS, HRMS, or HCM systems
- Maintaining employee records in a human resources system
- Configuring Workday or similar HR platforms
- Creating HR reports and dashboards
- Protecting employee data and following privacy requirements
- Supporting payroll, benefits, recruiting, and employee lifecycle data

This role is important because human resources departments manage large amounts of sensitive employee data. Information systems help HR employees maintain accurate records, support payroll and benefits, track workforce changes, and provide reports to managers.

5.5 Preparation for These Jobs

To prepare for these jobs, a student should develop both business knowledge and information systems skills. The most important preparation steps are:

- Learn spreadsheet analysis, including formulas, pivot tables, lookup functions, and charts.
- Learn database concepts, including tables, primary keys, foreign keys, joins, grouping, and SQL queries.
- Practice using business intelligence tools such as Power BI or Tableau.
- Understand major enterprise systems, including ERP, CRM, and HRIS platforms.
- Build small portfolio projects that show the ability to clean data, query data, analyze trends, and present results.
- Develop communication skills so technical findings can be explained to business managers.

This MIS project helps prepare for these jobs because it uses a real transactional database, converts data into usable formats, applies SQL queries, and summarizes results for management decision making.

5.6 Summary of Job Search Findings

The job search shows that information systems knowledge is required in many business careers, not only in IT departments. Accounting and finance roles use ERP systems, spreadsheets, and reporting tools. Sales operations roles use CRM, SAP, SQL, and business intelligence dashboards. Human resources roles use HRIS and Workday systems to manage employee information and reporting.

Overall, the most useful information systems skills for business students are spreadsheet analysis, database querying, business intelligence reporting, enterprise system knowledge, and data-driven decision making.

5.7 Sources Consulted

- MUFG Accounting Transaction Analyst job posting: https://mufgub.wd3.myworkdayjobs.com/en-US/MUFG-Careers/job/Accounting-Transaction-Analyst_10072732-WD
- Monster financial analyst job search results: <https://www.monster.com/jobs/q-financial-analys>
- ZEISS Sales Operations Analyst job posting: https://zeissgroup.wd3.myworkdayjobs.com/en-US/External/job/Analyst-II--Sales-Operations_JR_1049741
- Monster HRIS Analyst job search results: <https://www.monster.com/jobs/q-hris-analyst-jobs>

- O*NET Human Resources Specialists: <https://www.onetonline.org/link/summary/13-1071.00>
- O*NET Market Research Analysts Technology Skills: <https://www.onetonline.org/link/tt/13-1161.00/43232306>
- BLS Market Research Analysts Occupational Outlook: <https://www.bls.gov/ooh/business-and-financial/market-research-analysts.htm>

6 Conclusion

This project shows how information systems can improve business performance, management decision making, and operational control. The Snyder's of Hanover case demonstrated the problems created by manual spreadsheet-based reporting. Slow data collection, repeated manual entry, and inconsistent spreadsheet updates can delay financial reporting and reduce the quality of management decisions. A centralized financial information system or ERP system would help the company produce faster, more accurate, and more useful reports.

The Dollar General case showed that inventory management problems should not be solved by purchasing technology without first understanding the business need. Before investing in an information system, management must define the problem, identify required system features, evaluate costs and benefits, plan employee training, and consider security requirements. An automated inventory management system using barcode scanning and a centralized database could reduce inventory losses and improve store-level decision making.

The database analysis project demonstrated how raw transactional data can be converted into useful management information. SQL queries were used to analyze product demand, regional sales performance, seller performance, and monthly sales trends. The analysis found that product categories such as `cama_mesa_banho`, `beleza_saude`, `esporte_lazer`, `moveis_decoracao`, and `informatica_acessorios` should be prioritized for restocking because they had the highest sales volume. The analysis also showed that lower-performing regions such as AM, PA, SE, PI, and RO may benefit from additional marketing or seller expansion. Finally, the monthly sales analysis showed that stronger sales months such as March, April, May, July, and August are better candidates for full-price selling, while weaker months such as September, October, December, November, and January may require discounts or promotional campaigns.

The job search project showed that information systems knowledge is important in many business careers, not only in technical IT positions. Accounting, finance, sales, marketing, and human resources jobs increasingly require skills in spreadsheets, databases, ERP systems, CRM systems, HRIS platforms, SQL, and business intelligence tools.

Overall, this project demonstrates that information systems help organizations collect data, organize data, analyze trends, and make better decisions. Managers who understand information systems can improve efficiency, reduce errors, control costs, identify opportunities, and respond more quickly to business problems.

A Appendix A: SQLite to CSV Export Script

The following Python script was used to export the SQLite database tables into CSV files. This made the data easier to inspect and made it possible to use the data in spreadsheet or database software if needed.

Listing 1: SQLite to CSV Export Script

```
1  #!/usr/bin/env python3
2
3  import sqlite3
4  import pandas as pd
5  import os
6  import argparse
7
8
9  def export_sqlite_to_csv(database_path, output_folder):
10     os.makedirs(output_folder, exist_ok=True)
11
12     conn = sqlite3.connect(database_path)
13
14     # Get all tables
15     tables = pd.read_sql_query(
16         """
17         SELECT name
18         FROM sqlite_master
19         WHERE type='table'
20         ORDER BY name;
21         """,
22         conn
23     )
24
25     for table in tables["name"]:
26         print(f"Exporting table: {table}")
27
28         df = pd.read_sql_query(
29             f"SELECT * FROM {table}",
30             conn
31         )
32
33         output_file = os.path.join(
34             output_folder,
35             f"{table}.csv"
36         )
37
38         df.to_csv(
39             output_file,
40             index=False
41         )
```

```

42
43         print(
44             f"Saved {len(df)} rows -> {output_file}"
45         )
46
47     conn.close()
48
49     print("\nExport completed successfully.")
50
51
52 if __name__ == "__main__":
53
54     parser = argparse.ArgumentParser(
55         description="Export SQLite database tables to CSV files"
56     )
57
58     parser.add_argument(
59         "database",
60         help="Path to SQLite database file"
61     )
62
63     parser.add_argument(
64         "output",
65         help="Folder where CSV files will be created"
66     )
67
68     args = parser.parse_args()
69
70     export_sqlite_to_csv(
71         args.database,
72         args.output
73     )

```

B Appendix B: Database Profile and Base View SQL

The following SQL file was used to inspect the database, count records, check the date range, and create the main joined sales analysis view.

Listing 2: Database Profile and Sales Analysis Base View

```
1 .headers on
2 .mode column
3
4 /*
5 Hands-On MIS Sales Analysis Queries
6
7 This file prepares the dataset for the Chapter 1 MIS database
8 analysis project.
9 The main goal is to answer:
10
11 1. Which products should be restocked?
12 2. Which stores and sales regions need promotional campaigns?
13 3. When should products be sold at full price or discounted?
14 */
15
16 /*
17 SECTION 1: Database Profile
18 */
19
20 SELECT 'orders' AS table_name, COUNT(*) AS row_count FROM orders
21 UNION ALL
22 SELECT 'order_items', COUNT(*) FROM order_items
23 UNION ALL
24 SELECT 'products', COUNT(*) FROM products
25 UNION ALL
26 SELECT 'sellers', COUNT(*) FROM sellers
27 UNION ALL
28 SELECT 'order_reviews', COUNT(*) FROM order_reviews;
29
30
31 /*
32 SECTION 2: Date Range
33 */
34
35 SELECT
36     MIN(timestamp) AS first_order_date,
37     MAX(timestamp) AS last_order_date
38 FROM orders;
39
40
41 /*
```

```

42 SECTION 3: Main Sales Analysis View
43
44 Each row in order_items represents one sold item.
45 So COUNT(*) can be used as units_sold.
46 */
47
48 DROP VIEW IF EXISTS sales_analysis_base;
49
50 CREATE VIEW sales_analysis_base AS
51 SELECT
52     oi.order_items_pk,
53     o.order_id,
54     DATE(o.timestamp) AS order_date,
55     STRFTIME('%Y', o.timestamp) AS order_year,
56     STRFTIME('%m', o.timestamp) AS order_month,
57     STRFTIME('%Y-%m', o.timestamp) AS order_year_month,
58     oi.product_id,
59     p.product_category_name,
60     oi.seller_id,
61     s.seller_state,
62     oi.price
63 FROM order_items AS oi
64 JOIN orders AS o
65     ON oi.order_id = o.order_id
66 LEFT JOIN products AS p
67     ON oi.product_id = p.product_id
68 LEFT JOIN sellers AS s
69     ON oi.seller_id = s.seller_id;
70
71
72 /*
73 SECTION 4: View Check
74 */
75
76 SELECT
77     COUNT(*) AS total_sales_rows
78 FROM sales_analysis_base;
79
80
81 /*
82 SECTION 5: Sample Rows
83 */
84
85 SELECT
86     order_date,
87     order_year_month,
88     seller_state,

```



```

89     product_category_name ,
90     price
91 FROM sales_analysis_base
92 LIMIT 10;
93
94
95 /*
96 SECTION 6: Missing Data Check
97 */
98
99 SELECT
100     COUNT(*) AS total_rows ,
101     SUM(CASE WHEN product_category_name IS NULL THEN 1 ELSE 0 END)
        AS missing_product_category ,
102     SUM(CASE WHEN seller_state IS NULL THEN 1 ELSE 0 END) AS
        missing_seller_state ,
103     SUM(CASE WHEN price IS NULL THEN 1 ELSE 0 END) AS missing_price
104 FROM sales_analysis_base;

```

C Appendix C: Restocking Analysis SQL

The following SQL file was used to identify high-demand product categories and products that should be prioritized for restocking.

Listing 3: Restocking Analysis SQL Queries

```
1 .headers on
2 .mode column
3
4 /*
5  Restocking Analysis
6
7  Purpose:
8  Identify product categories and individual products with the highest
9    sales demand.
10
11 Business logic:
12 Because the dataset does not include current inventory levels, units
13   sold are used
14 as a demand indicator. Products with high units sold should be
15   prioritized for
16   restocking.
17 */
18
19 /*
20  Query 1A:
21  Top product categories by units sold.
22 */
23
24 SELECT
25     product_category_name AS product_category,
26     COUNT(*) AS units_sold,
27     ROUND(SUM(price), 2) AS total_revenue,
28     ROUND(AVG(price), 2) AS average_price
29 FROM sales_analysis_base
30 GROUP BY product_category_name
31 ORDER BY units_sold DESC
32 LIMIT 15;
33
34 /*
35  Query 1B:
36  Top individual products by units sold.
37 */
38
39 SELECT
40     product_id,
```

```

40     product_category_name AS product_category,
41     COUNT(*) AS units_sold,
42     ROUND(SUM(price), 2) AS total_revenue,
43     ROUND(AVG(price), 2) AS average_price
44 FROM sales_analysis_base
45 GROUP BY product_id, product_category_name
46 ORDER BY units_sold DESC
47 LIMIT 15;
48
49
50 /*
51 Query 1C:
52 High-revenue product categories.
53 This helps managers identify products that are not only popular,
54 but also financially important.
55 */
56
57 SELECT
58     product_category_name AS product_category,
59     COUNT(*) AS units_sold,
60     ROUND(SUM(price), 2) AS total_revenue,
61     ROUND(AVG(price), 2) AS average_price
62 FROM sales_analysis_base
63 GROUP BY product_category_name
64 ORDER BY total_revenue DESC
65 LIMIT 15;

```

D Appendix D: Promotional Campaign Analysis SQL

The following SQL file was used to identify low-performing regions and sellers that may benefit from promotional campaigns or additional marketing support.

Listing 4: Promotional Campaign Analysis SQL Queries

```
1 .headers on
2 .mode column
3
4 /*
5  Promotional Campaign Analysis
6
7  Purpose:
8  Identify stores and sales regions that may benefit from additional
9    marketing.
10
11 Business logic:
12 Low-performing regions and sellers may need promotional campaigns,
13   better
14 marketing support, or pricing strategy changes.
15
16 In this dataset:
17 - seller_id represents a store/seller
18 - seller_state represents a sales region
19 */
20
21 /*
22  Recreate the base view to make this SQL file runnable independently.
23 */
24 DROP VIEW IF EXISTS sales_analysis_base;
25
26 CREATE VIEW sales_analysis_base AS
27 SELECT
28     oi.order_items_pk,
29     o.order_id,
30     DATE(o.timestamp) AS order_date,
31     STRFTIME('%Y', o.timestamp) AS order_year,
32     STRFTIME('%m', o.timestamp) AS order_month,
33     STRFTIME('%Y-%m', o.timestamp) AS order_year_month,
34     oi.product_id,
35     p.product_category_name,
36     oi.seller_id,
37     s.seller_state,
38     oi.price
39 FROM order_items AS oi
40 JOIN orders AS o
```

```

41         ON oi.order_id = o.order_id
42 LEFT JOIN products AS p
43         ON oi.product_id = p.product_id
44 LEFT JOIN sellers AS s
45         ON oi.seller_id = s.seller_id;
46
47
48 /*
49 Query 2A:
50 Sales performance by region.
51 This shows total sales activity by seller state.
52 */
53
54 SELECT
55     seller_state AS region,
56     COUNT(*) AS units_sold,
57     COUNT(DISTINCT seller_id) AS number_of_sellers,
58     ROUND(SUM(price), 2) AS total_revenue,
59     ROUND(AVG(price), 2) AS average_price
60 FROM sales_analysis_base
61 GROUP BY seller_state
62 ORDER BY total_revenue ASC;
63
64
65 /*
66 Query 2B:
67 Lowest-performing regions by revenue.
68
69 These regions are candidates for additional marketing because their
70 total sales
71 revenue is low compared with other regions.
72 */
73 SELECT
74     seller_state AS region,
75     COUNT(*) AS units_sold,
76     COUNT(DISTINCT seller_id) AS number_of_sellers,
77     ROUND(SUM(price), 2) AS total_revenue,
78     ROUND(AVG(price), 2) AS average_price
79 FROM sales_analysis_base
80 GROUP BY seller_state
81 ORDER BY total_revenue ASC
82 LIMIT 10;
83
84
85 /*
86 Query 2C:

```

```

87 Highest-performing regions by revenue.
88
89 This gives a comparison point for identifying stronger and weaker
90 regions.
91 */
92 SELECT
93     seller_state AS region,
94     COUNT(*) AS units_sold,
95     COUNT(DISTINCT seller_id) AS number_of_sellers,
96     ROUND(SUM(price), 2) AS total_revenue,
97     ROUND(AVG(price), 2) AS average_price
98 FROM sales_analysis_base
99 GROUP BY seller_state
100 ORDER BY total_revenue DESC
101 LIMIT 10;
102
103
104 /*
105 Query 2D:
106 Lowest-performing sellers with at least 20 units sold.
107
108 The filter avoids sellers with only one or two sales, because those
109 may not be
110 meaningful enough for a promotional campaign.
111 */
112 SELECT
113     seller_id,
114     seller_state AS region,
115     COUNT(*) AS units_sold,
116     ROUND(SUM(price), 2) AS total_revenue,
117     ROUND(AVG(price), 2) AS average_price
118 FROM sales_analysis_base
119 GROUP BY seller_id, seller_state
120 HAVING units_sold >= 20
121 ORDER BY total_revenue ASC
122 LIMIT 15;
123
124
125 /*
126 Query 2E:
127 Highest-performing sellers.
128
129 This helps compare low-performing sellers with successful sellers.
130 */
131

```

```
132 SELECT
133     seller_id,
134     seller_state AS region,
135     COUNT(*) AS units_sold,
136     ROUND(SUM(price), 2) AS total_revenue,
137     ROUND(AVG(price), 2) AS average_price
138 FROM sales_analysis_base
139 GROUP BY seller_id, seller_state
140 ORDER BY total_revenue DESC
141 LIMIT 15;
```

E Appendix E: Pricing and Discount Timing SQL

The following SQL file was used to analyze monthly sales trends and identify stronger and weaker sales periods for pricing and promotion decisions.

Listing 5: Pricing and Discount Timing SQL Queries

```
1 .headers on
2 .mode column
3
4 /*
5 Pricing Timing Analysis
6
7 Purpose:
8 Identify when products should be offered at full price and when
9 discounts
10 or promotions should be used.
11
12 Business logic:
13 The dataset does not contain a discount field. Therefore, sales
14 volume and
15 monthly revenue are used to identify high-demand and low-demand
16 periods.
17
18 High-demand months:
19 - stronger sales volume
20 - higher revenue
21 - better candidates for full-price selling
22
23 Low-demand months:
24 - weaker sales volume
25 - lower revenue
26 - better candidates for discounts and promotions
27 */
28
29 /*
30 Recreate the base view so this file can run independently.
31 */
32
33 DROP VIEW IF EXISTS sales_analysis_base;
34
35 CREATE VIEW sales_analysis_base AS
36 SELECT
37     oi.order_items_pk,
38     o.order_id,
39     DATE(o.timestamp) AS order_date,
40     STRFTIME('%Y', o.timestamp) AS order_year,
41     STRFTIME('%m', o.timestamp) AS order_month,
```



```

40     STRFTIME('%Y-%m', o.timestamp) AS order_year_month,
41     oi.product_id,
42     p.product_category_name,
43     oi.seller_id,
44     s.seller_state,
45     oi.price
46 FROM order_items AS oi
47 JOIN orders AS o
48     ON oi.order_id = o.order_id
49 LEFT JOIN products AS p
50     ON oi.product_id = p.product_id
51 LEFT JOIN sellers AS s
52     ON oi.seller_id = s.seller_id;
53
54
55 /*
56 Query 3A:
57 Sales by month across the whole dataset.
58 */
59
60 SELECT
61     order_month AS month_number,
62     COUNT(*) AS units_sold,
63     ROUND(SUM(price), 2) AS total_revenue,
64     ROUND(AVG(price), 2) AS average_price
65 FROM sales_analysis_base
66 GROUP BY order_month
67 ORDER BY month_number;
68
69
70 /*
71 Query 3B:
72 Best months by total revenue.
73 These months are stronger candidates for full-price selling.
74 */
75
76 SELECT
77     order_month AS month_number,
78     COUNT(*) AS units_sold,
79     ROUND(SUM(price), 2) AS total_revenue,
80     ROUND(AVG(price), 2) AS average_price
81 FROM sales_analysis_base
82 GROUP BY order_month
83 ORDER BY total_revenue DESC
84 LIMIT 5;
85
86

```

```

87  /*
88  Query 3C:
89  Weakest months by total revenue.
90  These months are stronger candidates for discounts or promotions.
91  */
92
93  SELECT
94      order_month AS month_number,
95      COUNT(*) AS units_sold,
96      ROUND(SUM(price), 2) AS total_revenue,
97      ROUND(AVG(price), 2) AS average_price
98  FROM sales_analysis_base
99  GROUP BY order_month
100 ORDER BY total_revenue ASC
101 LIMIT 5;
102
103
104  /*
105  Query 3D:
106  Sales by year-month for more detailed time trend analysis.
107  */
108
109  SELECT
110      order_year_month,
111      COUNT(*) AS units_sold,
112      ROUND(SUM(price), 2) AS total_revenue,
113      ROUND(AVG(price), 2) AS average_price
114  FROM sales_analysis_base
115  GROUP BY order_year_month
116  ORDER BY order_year_month;
117
118
119  /*
120  Query 3E:
121  Strongest individual year-month periods.
122  */
123
124  SELECT
125      order_year_month,
126      COUNT(*) AS units_sold,
127      ROUND(SUM(price), 2) AS total_revenue,
128      ROUND(AVG(price), 2) AS average_price
129  FROM sales_analysis_base
130  GROUP BY order_year_month
131  ORDER BY total_revenue DESC
132  LIMIT 10;
133

```

```

134
135 /*
136 Query 3F:
137 Weakest individual year-month periods.
138
139 The very beginning and ending months may be incomplete because the
140 dataset
141 does not cover full calendar years, so these should be interpreted
142 carefully.
143 */
144
145 SELECT
146     order_year_month,
147     COUNT(*) AS units_sold,
148     ROUND(SUM(price), 2) AS total_revenue,
149     ROUND(AVG(price), 2) AS average_price
150 FROM sales_analysis_base
151 GROUP BY order_year_month
152 ORDER BY total_revenue ASC
153 LIMIT 10;

```